

Pluggable Device Font Renderer

Architecture, integration, and module reference

Project	Ruffle - Flash Player emulator (Rust + WebAssembly + TypeScript)
Branch	pluggable-font-renderer
Status	Working draft, prepared after the swash-renderer rendering fix
Audience	Maintainers, integrators, third-party renderer authors
Scope	Web platform - any JavaScript host able to load a bridge module (browsers, Electron, and similar embedders).

This document describes the existing Ruffle device font rendering pipeline, the additive integration that allows external renderers to be plugged in at runtime, the contract those renderers must honour, and the two reference implementations developed alongside the contract: **gdi-font-renderer** (Windows GDI via napi-rs) and **swash-font-renderer** (pure-WebAssembly via swash).

Table of contents

1. Ruffle font rendering architecture, before and after	4
1.1 Glossary	4
1.2 Pipeline overview	5
1.3 Cache strategy and intrinsic_scale	6
1.4 Existing built-in renderers	9
2. Integrating a custom renderer into Ruffle	10
2.1 core/src/font.rs - extend the FontRenderer trait	10
2.2 web/src/lib.rs - new DeviceFontRenderer variant	10
2.3 web/src/builder.rs - recognise "custom"	11
2.4 web/src/ui.rs - dispatch in load_device_font	11
2.5 web/src/ui/font_renderer.rs - the bridge proxy	12
2.6 web/packages/core/src/internal/custom-font-bridge.ts	13
2.7 web/packages/core/src/public/config/custom-font-bridge.ts and load-options.ts	14
2.8 web/packages/core/src/internal/builder.ts	14
2.9 web/packages/core/src/internal/player/inner.tsx	15
2.10 Cargo.toml - workspace	15
3. The pluggable module contract	16
3.1 Technology is up to the module author	16
3.2 The shared FontBridge contract	16
3.3 The CustomFontRenderer instance contract	17
3.4 How a bridge is registered	17
4. The gdi-font-renderer module	19
4.1 Module shape	19
4.2 Architecture	19
4.3 Rasterization details	19
4.4 hasKerningInfo policy	20
4.5 Reported interface (auto-generated by napi-rs)	20
4.6 Usage	21
5. The swash-font-renderer module	22
5.1 Module shape	22

5.2 Architecture	22
5.3 registerFontData - face indexing	22
5.4 createRenderer and the SizedFont cache	23
5.5 renderGlyph - rasterization with bitmap padding	23
5.6 calculateKerning	24
5.7 Lifecycle and memory	24
5.8 Usage	25
Appendix A. Future work: the JS / WASM boundary	26
A.1 What each boundary crossing costs	26
A.2 Why it is not a problem today	26
A.3 When it might be worth optimizing	26

1. Ruffle font rendering architecture, before and after

Ruffle's text pipeline is split into three layers that each own a different concern: the SWF interpreter and ActionScript layout in **ruffle_core**, a font abstraction that hides the difference between embedded (SWF-supplied) fonts and host ("device") fonts in **ruffle_core::font**, and a per-platform **UiBackend** implementation that decides where unknown device fonts come from. On the web, that backend is **WebUiBackend**.

This three-layer split is pre-existing, and the present branch does **not** change it. What does change is the set of rasterizers that **WebUiBackend** can dispatch to. On mainline Ruffle there are two: Embedded (bitmaps from SWF-provided outlines or from TTFs passed via `fontSources`) and Canvas (`OffscreenCanvas.measureText/fillText`). This branch adds a third variant, Custom, that routes every per-glyph call to a JS-side bridge object installed by the embedding host on `globalThis.__ruffleCustomFontRenderer` - the *pluggable renderer* that gives this document its name. The branch also teaches the shared glyph cache how to accommodate pixel-locked backends that cannot reuse a single bitmap across sizes.

The rest of this chapter inventories the pipeline that already exists (§1.1 glossary, §1.2 overview diagram with a before/after comparison), then describes the cache rework that makes the new variant viable (§1.3) and lists the two built-in rasterizers the branch inherits from mainline (§1.4). Chapters 2+ cover the integration itself.

1.1 Glossary

Term	Meaning
Twips	Internal Flash unit. 1 px = 20 twips. Most metrics are stored as <i>twips</i> to keep the rasterizer free of floating-point drift.
FontQuery	{name, is_bold, is_italic} tuple the layout uses to look up a font.
FontMetrics	{scale, ascent, descent, leading} in twips. scale is the em-size at which the bitmap was rasterized.
Glyph	Per-codepoint result. Holds either a vector outline or a rasterized bitmap, plus an advance and an optional <code>intrinsic_scale</code> .
FontType	Embedded when the SWF brought its own outlines, Device when Ruffle had to ask the host.
DefaultFont	Symbolic mapping for Flash's <code>_sans</code> , <code>_serif</code> , <code>_typewriter</code> , and the Japanese defaults.
FontRenderer	Trait implemented by every device-font backend. Owns metrics and per-glyph rasterization.
DeviceFontRenderer	Web-only enum chosen at builder time: Embedded, Canvas, or Custom. The bridge object for Custom is looked up lazily on <code>globalThis.__ruffleCustomFontRenderer</code> .

1.2 Pipeline overview

Common to both mainline and this branch. When the AVM (the SWF interpreter) needs to draw text, `EditText` in `core/src/html` walks the run of formatted text, asks the resident **Font** for each glyph at the requested height, and feeds the resulting `GlyphRefs` to the active render backend. If the requested font is unknown - because no SWF tag defined it and no previous lookup cached it - the layout falls back to the Player's `load_device_font()` hook, which the web build delegates to `WebUiBackend::load_device_font`. That method inspects the chosen `DeviceFontRenderer` variant and registers a `FontDefinition` that carries the appropriate `FontRenderer` implementation. From that point on the layout loop treats the font like any other: metrics come from the trait, bitmaps from per-glyph calls, and the cache in `ruffle_core::font::Font` makes sure no glyph is rasterized twice.

What this branch changes. The diagram on the right below highlights in orange the three boxes - and the two arrow labels - that this branch touches:

- **FontRenderer trait impl:** mainline accepts only `Embedded` and `Canvas`. The branch adds `Custom`, which looks up a JS-side bridge object on `globalThis.__ruffleCustomFontRenderer` and forwards all per-glyph calls to it (see §2.5 and Chapter 3).
- **ruffle_core::font::Font cache:** the cache key grows from a single code point to a `(code_point, size_px)` tuple. Backends that do not need size-specific rasters (`Canvas`, `Embedded`) keep sharing a single bucket via `cache_size = 0`; size-aware backends (`GDI`) pay for the extra entries. Detailed in §1.3.
- **Render backend:** pixel-locked glyphs now carry an optional `intrinsic_scale` so the layout can draw them 1:1 to device pixels instead of rescaling. Mainline behavior is preserved for glyphs that leave the field `None`.
- **Per-glyph trait calls:** the size-agnostic `render_glyph / calculate_kerning` are kept as defaults; new `_at_size` overloads let backends opt in to size-aware rasterization. The enum dispatch that makes this possible is documented in §2.1.

The top four boxes in the diagram are unchanged: the SWF interpreter, the `EditText` layout, the `load_device_font` hook, and the `WebUiBackend` dispatcher all behave exactly as before. The branch is deliberately additive - any code path that does not ask for a `Custom` renderer produces the same bitmaps on this branch as on mainline.

Before (mainline Ruffle)

device-font pipeline without pluggable renderers

After (this branch)

Custom variant + `__ruffleCustomFontRenderer` global slot,
size-aware cache, `intrinsic_scale`



Figure 1. The device-font pipeline, mainline (left) versus this branch (right). Orange header strips and bold-oblique arrow labels mark the boxes and trait calls that the branch modifies; everything else is identical by design.

1.3 Cache strategy and intrinsic_scale

The per-glyph cache inside `ruffle_core::font::Font` is the pivot around which the whole pluggable renderer contract is designed. Ruffle already had a cache before this branch, but it was built on an assumption - *one bitmap per code point is enough* - that pixel-locked backends like GDI cannot honour. This subsection explains the state of the cache on mainline, what broke when a GDI-style renderer was plugged in, and the size-aware rework that makes both worlds coexist without cost to the existing paths.

1.3.1 The original size-agnostic cache

On mainline Ruffle, the `ExternalRenderer` variant of `GlyphSource` stored two `RefCell<HashMap>`s:

```
// Before (mainline)
ExternalRenderer {
    glyph_cache: RefCell<FnvHashMap<u16, Option<Glyph>>>,
    kerning_cache: RefCell<FnvHashMap<(u16, u16), Twips>>,
    font_renderer: Box<dyn FontRenderer>,
},
```

`get_by_code_point(&self, code_point: char)` took no size argument, and the trait had a single `render_glyph(char) -> Option<Glyph>` method. The implicit contract: the renderer rasterizes once at a **canonical em-size** (64 px for the existing canvas backend), and the layout code in `Font::evaluate_layout_operation` computes a runtime scale factor:

```
// Before (mainline layout loop)
let scale = params.height.get() as f32 / resolution.font.scale();
```

Multiplying every glyph vertex / bitmap by `scale` happens on the GPU side as part of the normal draw transform. The cache could therefore be keyed solely by code point, and memory usage stayed bounded regardless of how many different font sizes a movie used: one bitmap served them all.

Why this worked: the canvas backend uses the browser's vector `fillText` pipeline - scaling a 64-pixel-em rasterization to a 12-pixel run produces a visually acceptable result (the browser runs its own hinting internally). The embedded SWF path feeds vector outlines to the GPU, which scales them without any quality loss at all. Neither path needed per-size storage.

1.3.2 Why this broke for pixel-locked backends

The defining property of a GDI-style backend is that the *only* way to get a crisp glyph is to ask the OS to rasterize it at the exact display size - with its own hinting, its own stem snapping, and the platform's native antialiasing strategy (ClearType on Windows). The moment the layout pipeline scales such a bitmap by any factor other than 1.0, bilinear filtering smears the subpixel-tuned output into the blur the renderer was supposed to avoid.

In other words: to keep GDI output pixel-perfect, the bitmap must reach the framebuffer with `scale = 1.0`. That requires two things the mainline cache did **not** provide:

- **A separate cache entry per display size**, so that a 12 px render and a 24 px render of the same code point do not collide on the same HashMap key.
- **A way for a bitmap to carry its own em-scale**, so that the layout divisor cancels out and the runtime scale collapses to 1.0 rather than stretching the bitmap to match the font's canonical size.

A naive fix - always key the cache by `(code_point, size_px)` - would have solved the GDI case at a substantial memory cost for the canvas and embedded paths: the same glyph would have been re-stored for every requested size, even though the bitmap bytes are identical modulo GPU scaling. A Flash movie that mixes a dozen font sizes on screen would have inflated the cache by the same factor for no visible benefit.

1.3.3 The size-aware rework, free for existing backends

The redesign splits the decision of *how to key the cache* away from the cache data structure itself, and lets each backend opt in. Three changes to the trait:

```
pub trait FontRenderer: std::fmt::Debug {
    // Unchanged size-agnostic hooks (used by canvas / embedded):
    fn render_glyph(&self, character: char) -> Option<Glyph>;
    fn calculate_kerning(&self, left: char, right: char) -> Twips;

    // New: opt-in size-aware variants, default-delegate.
    fn render_glyph_at_size(&self, character: char, _height_px: u32)
        -> Option<Glyph> { self.render_glyph(character) }

    fn calculate_kerning_at_size(&self, left: char, right: char,
                               _height_px: u32) -> Twips {
```

```

        self.calculate_kerning(left, right)
    }

    // New: tells the cache whether to key by size.
    fn is_size_aware(&self) -> bool { false }
}

```

The cache then extends its key tuples with a `height_px` component and picks the effective key at lookup time based on the backend's own flag:

```

// After (this branch)
ExternalRenderer {
    glyph_cache: RefCell<FnvHashMap<(u16, u32), Option<Glyph>>>,
    kerning_cache: RefCell<FnvHashMap<(u16, u16, u32), Twips>>,
    font_renderer: Box<dyn FontRenderer>,
}

let cache_size = if font_renderer.is_size_aware() {
    height_px // one entry per display size
} else {
    0 // every size collapses onto the same bucket
};
glyph_cache.borrow_mut()
    .entry((code_point, cache_size))
    .or_insert_with(|| font_renderer.render_glyph_at_size(ch, cache_size));

```

The `cache_size = 0` branch is the whole point of the design: size-unaware backends keep using a single bucket per code point, so **canvas and embedded fonts pay zero extra memory** after the rework. The only observable change for them is the extra `u32` in the tuple key - a few bytes per glyph actually cached - and an unused `_height_px` argument that the default trait methods discard.

Two auxiliary pieces make this end-to-end:

- **Twips-to-pixel quantization.** A small helper `twips_to_px_for_cache(Twips) -> u32` converts requested heights with half-pixel rounding ($((twips + 10) / 20)$, clamped to at least 1). Without quantization, floating-point noise in layout-time scale computations would fragment the cache into near-duplicate entries at 11.97 px, 12.00 px, 12.03 px, etc.
- **Height plumbed through the lookup APIs.** `Font::get_glyph_for_char`, `GlyphSource::get_by_code_point`, `FontLike::resolve_glyph` and `get_kerning_offset` all gained a `height: Twips` parameter, carried by the layout loop via `params.height`. Callers that only probe for existence (`has_glyphs_for_str`) pass `Twips::ZERO`, which canonicalizes to the shared `cache_size = 0` slot.

1.3.4 Glyph::intrinsic_scale and the 1:1 layout path

Cache keying alone is not enough to get a pixel-perfect draw: the layout also needs to stop rescaling the bitmap. **Glyph** gained an `intrinsic_scale: Option<f32>` field and a dedicated constructor `from_bitmap_at_scale` used by pixel-locked backends:

```

pub struct Glyph {
    shape: GlyphShape,
    advance: Twips,
    character: char,
    /// Em-size (in twips) at which this bitmap was rasterized.
    /// None falls back to the owning font's canonical scale.
    intrinsic_scale: Option<f32>,
}

```



```
}
```

The layout loop then consults it:

```
// After (this branch)
let glyph_scale_basis = glyph
    .intrinsic_scale()
    .unwrap_or_else(|| resolution.font.scale());
let scale = params.height.get() as f32 / glyph_scale_basis;
```

When a backend rasterizes at the display size and sets `intrinsic_scale = params.height`, the divisor cancels and `scale` collapses to 1.0 - every bitmap pixel lands on exactly one device pixel, with no bilinear blur. Backends that do not set the field (canvas, embedded, and any other size-unaware implementation) keep using the font's canonical scale exactly as before.

Net effect: two independent mechanisms - size-aware cache keying and `intrinsic_scale` - were added side by side. Each is a pure addition, guarded by a default that preserves the old behavior, and each solves half of the GDI problem. Together they make pluggable pixel-locked renderers feasible without regressing the mainline paths.

1.4 Existing built-in renderers

Two implementations ship in mainline Ruffle. They are listed below for context; the new Custom variant is described in detail in section 2.

Variant	Where the bitmap comes from	Notes
Embedded	SWF tags + any TTF/OTF/SWF passed via <code>fontSources</code>	Synchronous, registered up-front by the builder. Default.
Canvas	<code>OffscreenCanvas.measureText</code> / <code>fillText</code>	Browser does the rasterization; access to OS fonts via the CSS font system, but no glyph index, so kerning is approximated.

2. Integrating a custom renderer into Ruffle

The integration is deliberately additive: a single new variant is added to the existing `DeviceFontRenderer` enum, and the rest of the changes are the minimum plumbing needed to (a) carry that variant from JS configuration to the Rust UI backend and (b) bridge per-glyph calls across the JS/WASM boundary. No previous code path was removed: Embedded and Canvas still work exactly as before.

All file paths below are relative to the repository root. Each subsection states **what** changed and **why**.

2.1 core/src/font.rs - extend the `FontRenderer` trait

Two responsibilities were grafted onto the existing trait: size-aware rasterization (so backends that prefer to rasterize at the display size can do so), and a way for backends to tag their bitmaps with the em-scale they chose. The defaults are no-ops, so the canvas renderer is unaffected.

```
pub trait FontRenderer: std::fmt::Debug {
    fn scale(&self) -> f32;
    fn get_font_metrics(&self) -> FontMetrics;
    fn has_kerning_info(&self) -> bool;
    fn render_glyph(&self, character: char) -> Option<Glyph>;
    fn calculate_kerning(&self, left: char, right: char) -> Twips;

    fn render_glyph_at_size(&self, character: char, _height_px: u32)
        -> Option<Glyph> { self.render_glyph(character) }

    fn calculate_kerning_at_size(&self, left: char, right: char,
                               _height_px: u32) -> Twips {
        self.calculate_kerning(left, right)
    }

    fn is_size_aware(&self) -> bool { false }
}
```

Why: the existing single-cache contract was sufficient for the canvas renderer (which uses the browser's vector pipeline and scales without blur), but pixel-locked backends rasterize differently at every size. Adding new methods with default implementations keeps backwards compatibility while letting size-aware renderers opt in.

The `Glyph` struct also gained an `intrinsic_scale: Option<f32>` field and a constructor that sets it (from `bitmap_at_scale`). The layout code in `Font::evaluate_layout_operation` now consults it:

```
let glyph_scale_basis = glyph
    .intrinsic_scale()
    .unwrap_or_else(|| resolution.font.scale());
let scale = params.height.get() as f32 / glyph_scale_basis;
```

Why: when a backend rasterizes at the display size it produces a glyph whose em-scale equals `params.height`; the divisor cancels and scale collapses to 1.0, which means the GPU draws each bitmap pixel onto exactly one device pixel - no bilinear blur.

2.2 web/src/lib.rs - new `DeviceFontRenderer` variant

The enum that the WASM side carries through the builder gained a unit variant `Custom`. No payload: the bridge object is fetched lazily from a well-known JS global slot at every font request.

```
#[derive(Debug, Clone, PartialEq, Eq)]
```

```
pub enum DeviceFontRenderer {
    Embedded,
    Canvas,
    /// Delegates glyph rasterization to a pluggable bridge
    /// object installed on `globalThis.__ruffleCustomFontRenderer`.
    /// Fetched lazily on every font request.
    Custom,
}
```

Why: a unit variant keeps `RuffleInstanceBuilder` clonable without carrying any JS-side state. The bridge lives entirely in the JS global namespace and is read through reflection by `WebUiBackend`; this way the same bridge object can be shared across every player instance on the page, and bridges that load asynchronously (ES modules with top-level `await`) can be assigned after player construction without any further plumbing.

2.3 web/src/builder.rs - recognise "custom"

`setDeviceFontRenderer` accepts three string values and the builder skips its embedded TTF setup when a custom renderer is in use - otherwise the embedded path would compete with the bridge for the same family names.

```
#[wasm_bindgen(js_name = "setDeviceFontRenderer")]
pub fn set_device_font_renderer(&mut self, value: String) {
    self.device_font_renderer = match value.as_str() {
        "embedded" => DeviceFontRenderer::Embedded,
        "canvas"   => DeviceFontRenderer::Canvas,
        "custom"   => DeviceFontRenderer::Custom,
        _          => return,
    };
}

// Inside create_player:
if matches!(self.device_font_renderer, DeviceFontRenderer::Embedded) {
    self.setup_fonts(&mut core);
}
```

Why: the match is intentionally tolerant - an unknown value silently keeps the previous default rather than panicking, mirroring how the rest of `RuffleInstanceBuilder` handles configuration. The `setup_fonts` guard prevents Ruffle from registering its own TTF parses on top of the bridge: the bridge is the sole owner of the names it claims. Note that there is no `setter` for the bridge itself on this builder: the bridge lives on the JS global slot and is fetched by the UI backend, so the builder needs to carry no per-instance state.

2.4 web/src/ui.rs - dispatch in load_device_font

`WebUiBackend` took two changes: it now stores the chosen `DeviceFontRenderer` and dispatches on it inside `load_device_font`. The `Custom` arm looks up the bridge on the global slot at each call, then tries to instantiate the bridge-backed renderer - falling back to the canvas path if the slot is empty, the structural check fails, or the bridge itself rejects the family - so the player stays usable.

```
const CUSTOM_FONT_RENDERER_GLOBAL: &str = "__ruffleCustomFontRenderer";

fn lookup_custom_font_bridge() -> Option<JsValue> {
    let global = js_sys::global();
    let bridge =
        js_sys::Reflect::get(&global, &JsValue::from_str(CUSTOM_FONT_RENDERER_GLOBAL))
            .ok()?;
    if bridge.is_undefined() || bridge.is_null() {
```

```

        return None;
    }
    let create =
        js_sys::Reflect::get(&bridge, &JsValue::from_str("createRenderer")).ok()?;
    if !create.is_function() {
        return None;
    }
    Some(bridge)
}

fn load_device_font(&self, query: &FontQuery,
                    register: &mut dyn FnMut(FontDefinition)) {
    match &self.device_font_renderer {
        DeviceFontRenderer::Embedded => { /* registered up-front */ }
        DeviceFontRenderer::Canvas => register_canvas_font_renderer(query, register),
        DeviceFontRenderer::Custom => {
            let Some(bridge) = lookup_custom_font_bridge() else {
                tracing::warn!("...missing or invalid...");
                register_canvas_font_renderer(query, register);
                return;
            };
            match font_renderer::CustomFontRenderer::new(
                query.is_italic, query.is_bold, &query.name, &bridge,
            ) {
                Ok(r) => register(FontDefinition::ExternalRenderer { /* ... */ }),
                Err(e) => {
                    tracing::warn!("{}", e); // fall back to canvas
                    register_canvas_font_renderer(query, register);
                }
            }
        }
    }
}

```

Why lookup on every call: the bridge is produced and installed by the embedding host - Ruffle itself never attempts to load an addon - so the slot may still be empty at player construction and only materialize later (e.g. a `<script type="module">` with top-level `await` finishing after `DOMContentLoaded`). Reading `globalThis.__ruffleCustomFontRenderer` on every font request means any bridge that arrives before the first glyph is rendered is picked up automatically, without any extra coordination. Any number of host-side conditions can still leave the slot empty (the module failed to build, the host simply forgot to wire it up, etc.); in that case the soft fallback keeps the user-facing experience at "some text quality, no broken page".

2.5 web/src/ui/font_renderer.rs - the bridge proxy

`CustomFontRenderer` is the new `FontRenderer` implementation. It receives the bridge object as a `&JsValue` from `WebUiBackend::load_device_font` (which obtained it via `lookup_custom_font_bridge()` at the start of that call), invokes `createRenderer(family, bold, italic)` on it, caches the canonical-size metrics and the kerning-support flag, and forwards every per-glyph call to the JS object via reflection (`js_sys::Reflect`). The implementation is intentionally mechanical: any per-backend caching belongs to the addon, not to Ruffle.

Highlights of the proxy logic:

- **Bridge plumbing:** the constructor accepts the bridge as `&JsValue` (no global lookup from inside the renderer), calls `createRenderer` on it once, and stores the returned JS renderer object. Failure surfaces as a plain `JsValue` error caught by `WebUiBackend`.

- **Metrics cache:** `getFontMetrics` is invoked once at the canonical size (64 px) and the result is stored as twips, so steady-state operation never hits JS for metrics.
- **Per-glyph forwarding:** `renderGlyph(code, size_px)` returns `{ width, height, bitmapTx, advance, pixels }` or `null`; the proxy converts pixel fields to twips and produces `Glyph::from_bitmap_at_scale(...)` with `intrinsic_scale = size_px * 20`.
- **Kerning:** requests routed through `calculate_kerning_at_size`; the bridge is asked to convert pixel values back to twips on the Rust side, so the addon API stays in pixel units.
- **Lifecycle:** Drop calls `destroy()` on the JS object, ignoring errors (best effort).

Why this shape: the trait already has a notion of *caches*; pushing them down into the addon avoids growing Ruffle with addon-specific knowledge (HFONT pools, swash `ScaleContexts`, ttf-parser `Faces`). What stays in Rust is purely the JS plumbing.

2.6 web/packages/core/src/internal/custom-font-bridge.ts

This module owns the JS side of the bridge contract: the **FontBridge**, **CustomFontRenderer**, **FontMetrics** and **GlyphRaster** interfaces plus a handful of helpers used elsewhere in the player code. There is no JS-side registry object: the bridge lives directly on `globalThis.__ruffleCustomFontRenderer` and is read from there on demand.

Export	Used by	Role
<code>CUSTOM_FONT_RENDERER_GLOBAL</code>	<code>custom-font-bridge.ts</code> itself	The well-known name of the global slot, <code>"__ruffleCustomFontRenderer"</code> . Kept in sync with the constant of the same name in <code>web/src/ui.rs</code> so the TS and Rust sides never disagree on where to look.
<code>isFontBridge</code>	<code>custom-font-bridge.ts</code> itself	Structural check that the given value is an object with a <code>createRenderer</code> function. Everything else in the <code>FontBridge</code> contract is optional and probed later through reflection on the Rust side.
<code>getCustomFontRenderer</code>	<code>inner.tsx</code>	Reads <code>globalThis.__ruffleCustomFontRenderer</code> , runs it through <code>isFontBridge</code> , and returns the bridge or <code>undefined</code> . Each call re-reads the global, so bridges assigned after player construction are picked up as soon as they materialize.

Why a single global slot and not a registry: an earlier iteration kept a `__ruffleFontBridges` object keyed by module id so that multiple addons could coexist. That complexity bought nothing in practice - Ruffle only ever dispatches to one renderer per player anyway - and made the JS/Rust contract harder to reason about. The final shape is deliberately minimal: one JS global, one structural check, one lazy getter. Producing the bridge object remains entirely the embedding host's responsibility; Ruffle performs no dynamic loading of its own.

2.7 web/packages/core/src/public/config/custom-font-bridge.ts and load-options.ts

The public `@ruffle-rs/core` entry point needed one new module and a tiny extension to an existing enum. The re-export module exposes only the consumer-facing types - no helpers, because there is nothing for a third-party addon to call on the Ruffle side: the bridge is installed by direct assignment to the JS global. Meanwhile, `DeviceFontRenderer` gains a `Custom` variant that corresponds 1:1 to the Rust-side unit variant.

```
// public/config/custom-font-bridge.ts
export {
  type FontBridge,
  type CustomFontRenderer,
  type FontMetrics,
  type GlyphRaster,
} from "../../internal/custom-font-bridge";

// public/config/load-options.ts
export enum DeviceFontRenderer {
  Embedded = "embedded",
  Canvas   = "canvas",
  Custom   = "custom",
}

deviceFontRenderer?: DeviceFontRenderer;
```

Why: separating internal and public modules is the convention used throughout `web/packages/core`. Because the bridge is identified by a fixed global slot - not by a per-addon module id - the option value collapses to a plain enum with three discrete states, which is the smallest surface TypeScript users can be asked to type against. An earlier design used a literal-template type ``custom:${string}`` to carry the module id; that field is gone because no such id exists anymore.

2.8 web/packages/core/src/internal/builder.ts

`configureBuilder` already centralises the JS-to-Rust setting handoff. With the bridge now living on a single global slot that may be populated asynchronously, the JS side no longer tries to pre-validate the bridge - any such check would race the async loading it was meant to guard - and simply forwards "embedded", "canvas" or "custom" verbatim to Rust. The validation and soft fallback happen later, at font-request time, inside `load_device_font` (Section 2.4).

```
if (isExplicit(config.deviceFontRenderer)) {
  builder.setDeviceFontRenderer(config.deviceFontRenderer);
}
```

Why so short: the previous design parsed a `"custom:<id>"` prefix and looked the module id up in a JS-side registry object, downgrading to canvas whenever the lookup missed. That logic was removed together with the registry; the module id is no longer part of the option, and checking `globalThis.__ruffleCustomFontRenderer` at this exact moment would produce false negatives for bridges that legitimately arrive a few microtasks later (e.g. a `<script type="module">` that finishes initialising after `DOMContentLoaded`). Letting the Rust side own the lookup - once per font request, where the answer has to be right anyway - is both simpler and more accurate.

2.9 web/packages/core/src/internal/player/inner.tsx

Ruffle already fetches every URL from the `fontSources` array at startup, but until now the bytes were only pushed to the embedded path (`builder.addFont`). With Custom renderers added, `inner.tsx` also forwards the same bytes to the bridge - when one is present and implements `registerFontData` - so that bridges without access to an OS font database can build their face index from the same blob without an extra fetch.

```
if (this.loadedConfig?.fontSources) {
  const isCustom =
    this.loadedConfig.deviceFontRenderer === DeviceFontRenderer.Custom;
  for (const url of this.loadedConfig.fontSources) {
    try {
      const response = await fetch(url);
      const bytes = new Uint8Array(await response.arrayBuffer());
      builder.addFont(url, bytes);
      if (isCustom) {
        // Lazy lookup: a bridge that finishes its async
        // init mid-fetch still gets the bytes fetched after it.
        const bridge = getCustomFontRenderer();
        if (bridge?.registerFontData) {
          try {
            bridge.registerFontData(url, bytes);
          } catch (err) {
            console.warn(
              `Ruffle: __ruffleCustomFontRenderer threw ` +
              `from registerFontData("${url}")`, err,
            );
          }
        }
      }
    } catch (err) { /* fetch failure is logged */ }
  }
}
```

Why lazily in the loop: the bridge may be assigned to `globalThis.__ruffleCustomFontRenderer` by an async module that finishes loading while `inner.tsx` is still iterating the `fontSources` array. Capturing the bridge once outside the loop would leave later font sources stranded; re-reading the global inside each iteration closes that window. The alternative - having every such bridge fetch the URLs itself - would (a) duplicate the request and (b) push CORS configuration concerns onto the addon, so Ruffle remains the single I/O owner and the bridge merely receives bytes it has already paid for.

2.10 Cargo.toml - workspace

The new module crates live under `font-renderers/` at the repository root but are *not* added to the workspace members list, and they each define an empty `[workspace]` table in their own `Cargo.toml` so Cargo treats them as standalone workspaces. The motivation is twofold: their build targets differ from the main project (a custom `cdylib` on Windows for the napi addon, `wasm32-unknown-unknown` for swash) and their dependency closures pull in libraries the rest of Ruffle does not need.

3. The pluggable module contract

Everything Ruffle asks of a custom font renderer is spelled out by two TypeScript interfaces, **FontBridge** and **CustomFontRenderer**. A bridge object is a plain JavaScript value assigned to the well-known global slot `globalThis.__ruffleCustomFontRenderer`; Ruffle reads it from there on every font request and invokes methods by reflection. No part of this contract prescribes *how* the bridge is built - the implementation can be a native addon, a separate WebAssembly module, hand-rolled JavaScript, or anything else that meets the method signatures.

3.1 Technology is up to the module author

Ruffle is deliberately blind to the runtime nature of a bridge. `web/src/ui.rs` reads `globalThis.__ruffleCustomFontRenderer`, confirms the presence of a `createRenderer` function, and hands the resulting `JsValue` to the proxy in `web/src/ui/font_renderer.rs`, which then forwards per-glyph calls via `js_sys::Reflect`. Whatever lives on the other side of those reflective calls is entirely opaque to the player.

That opacity is the whole point of the mechanism: any reasonable way to produce a JS object that implements the contract is a valid pluggable renderer. The two reference modules that ship alongside this document - **gdi-font-renderer** (Chapter 4) and **swash-font-renderer** (Chapter 5) - are worked examples of two different technologies, not the canonical or exclusive choices:

- **gdi-font-renderer** is a `napi-rs` native Node addon that delegates rasterization to the Windows GDI text stack. It illustrates what a native addon looks like when it has access to an OS font database and platform-tuned rasterization.
- **swash-font-renderer** is a `wasm-bindgen` WebAssembly module that parses font bytes it receives from Ruffle and rasterizes them with the pure-Rust `swash` library. It illustrates what a portable, browser-only addon looks like when no OS integration is available.

Other plausible implementations include a native Core Text addon for macOS, a `fontconfig`-backed addon for Linux, a FreeType-based WASM module, a pure-JS wrapper around [Canvas 2D](#) that extracts its own glyphs, or even a stub used by tests to inject deterministic bitmaps. Ruffle treats all of them identically.

The practical consequences of the technology choice fall entirely on the addon author and the embedding host: build pipeline, platform coverage, whether font bytes need to be fed in via `registerFontData` (required whenever the addon cannot see the OS font store), and how the bridge object ends up on the registry. None of these choices leak into Ruffle's own code.

3.2 The shared FontBridge contract

The bridge object, regardless of how it is built, exposes:

```
interface FontBridge {
  createRenderer(
    family: string,
    bold: boolean,
    italic: boolean,
  ): CustomFontRenderer | null | undefined;

  // Optional. Implemented by bridges that need raw font bytes.
  registerFontData?(source: string, bytes: Uint8Array): void;
```



```
}

```

createRenderer is the only required method. Returning null or undefined tells Ruffle the bridge does not handle that family - the fallback to canvas is then taken on the Rust side. Throwing is treated as a hard error and surfaces as a one-time tracing warning.

registerFontData is invoked once per fontSources entry, after the bytes have been fetched, before the first createRenderer call. TTC/OTC collections are delivered in a single call - the bridge is responsible for iterating the faces inside. Any exception thrown is logged and swallowed; this method must never abort Ruffle's own font-source processing.

3.3 The CustomFontRenderer instance contract

```
interface CustomFontRenderer {
    hasKerningInfo(): boolean;
    getFontMetrics(sizePx: number): FontMetrics;
    renderGlyph(codePoint: number, sizePx: number): GlyphRaster | null;
    calculateKerning(left: number, right: number, sizePx: number): number;
    destroy(): void;
}

interface FontMetrics { ascent: number; descent: number; leading: number; }

interface GlyphRaster {
    width: number;      // bitmap width, >= 1
    height: number;     // bitmap height, >= 1
    bitmapTx: number;   // x offset of bitmap from pen, may be negative
    advance: number;    // pen advance after this glyph
    pixels: Uint8Array; // RGBA, row-major top-down, premultiplied
}
```

Conventions enforced by Ruffle:

- All numeric values are in **pixels** at sizePx. Ruffle converts to twips on the Rust side - addons never see the twips coordinate system.
- Bitmaps are **RGBA, top-down, row-major** with **premultiplied alpha**. Non-ink pixels are (0, 0, 0, 0). The buffer length must equal width * height * 4.
- Bitmap layout uses the canvas convention: height = ascent + descent, baseline at row y = ascent. The renderGlyph result must already be padded vertically; Ruffle does not adjust it.
- destroy() is called exactly once when Ruffle drops the renderer; the addon is expected to release every native handle it owns. Calling destroy() twice must be a no-op.

3.4 How a bridge is registered

There is one registration mechanism, regardless of what the addon is made of: the bridge object must sit on globalThis.__ruffleCustomFontRenderer before the first device-font glyph is requested. Ruffle performs no dynamic loading of its own - if the slot is empty or does not expose a createRenderer function when a font request arrives, load_device_font transparently falls back to the canvas renderer for that request (see Section 2.4), logs a warning, and the page keeps rendering.

The registration itself is a single JavaScript assignment:

```
globalThis.__ruffleCustomFontRenderer = bridge;
```

There is no helper to call and no object to allocate first: one slot, one assignment. The bridge value only needs to expose a `createRenderer(family, bold, italic)` function (plus the optional `registerFontData` hook); nothing else is inspected.

Because the slot is read lazily, the assignment can happen *before or after* the Ruffle player is constructed - what matters is that the bridge is in place by the time the first glyph has to be rasterized. This is what makes async bridges workable without ceremony: a `<script type="module">` can await its WASM init and write the slot on its own schedule, and Ruffle will pick it up on the next font request. How the *bridge object itself* is produced is entirely the host's concern. The two sketches below illustrate the ends of the spectrum; any other loading strategy is equally valid.

Example - Electron preload script (synchronous require):

```
// preload.js (runs with nodeIntegration available):
const bridge = require("@example/some-font-addon");
globalThis.__ruffleCustomFontRenderer = bridge;
```

Example - pure-browser ESM module (async import):

```
<script type="module">
  import init, * as bridge from "./some-font-addon.js";
  await init();
  globalThis.__ruffleCustomFontRenderer = bridge;
</script>
```

In the Electron snippet the assignment is synchronous and therefore unconditionally finishes before the renderer page runs any scripts. In the ESM snippet it is asynchronous, and may finish after the Ruffle player has already been constructed - that is fine: the lookup inside `load_device_font` happens per font request, so any bridge that lands before the first glyph is needed is honoured. If the bridge genuinely arrives too late (or never), Ruffle simply serves the affected fonts from the canvas path.

4. The gdi-font-renderer module

gdi-font-renderer is one of the two reference pluggable renderers shipped with this branch. It is a napi-rs native Node addon that delegates rasterization to the Windows GDI text stack, targeting Electron-based hosts on Windows where the goal is pixel-perfect, hinted output that matches the appearance of the native UI - the same look the original Flash Player produced for device fonts. Ruffle does not depend on it and is unaware of its implementation technology; what follows is an example of how a native-addon bridge is put together.

4.1 Module shape

Item	Value
Crate path	font-renderers/gdi-font-renderer
Crate name	ruffle-gdi-font-renderer
Build	napi build --release -> gdi-font-renderer.node
JS package	@ruffle-rs/gdi-font-renderer
Public exports	createRenderer(family, bold, italic): GdiFontRenderer and the GdiFontRenderer class
Platforms	Windows only. The addon calls directly into the Win32 GDI text stack, so only Windows targets compile. The napi CLI is invoked without --platform, which emits a single .node file with no architecture suffix - consumers require() it directly.

4.2 Architecture

The module is a single napi-rs class plus a thin factory. The class lazily creates a per-pixel-size **SizedFont**, each of which owns one HFONT. Per-size rasterization caches share a single Mutex<Inner> guard so napi-rs threads don't race on the GDI device context.

```
pub struct GdiFontRenderer {
    family: String,
    bold: bool,
    italic: bool,
    inner: Mutex<Inner>,
}

struct Inner {
    /// HDC bound to the screen, used as the rasterization device.
    hdc: HDC,
    /// One SizedFont per pixel size requested by Ruffle.
    sizes: HashMap<u32, SizedFont>,
}

struct SizedFont {
    hfont: HFONT,
    metrics: GdiFontMetrics,
    /// Glyph cache keyed by code point.
    glyphs: HashMap<u32, Option<GdiGlyphRaster>>,
}
```

4.3 Rasterization details

- **Font handle:** CreateFontW with `lfHeight = -size_px` (em-height in pixels), `lfWeight FW_BOLD` when bold, `lfItalic = TRUE` when italic. `CLEAR_TYPE_QUALITY` is requested.
- **Metrics:** read from `GetTextMetricsW` on the selected font. `tmAscent + tmDescent` becomes the bitmap height; `tmExternalLeading` becomes `FontMetrics.leading`.
- **Glyph bitmap:** `GetGlyphOutlineW` with `GGO_GRAY8_BITMAP` retrieves a 65-level gray ink mask plus a `GLYPHMETRICS` describing the black box and the cell origin. The mask is expanded to `RGBA` premultiplied, with alpha derived from the gray level.
- **Bitmap layout:** the bitmap is padded vertically to `height = ascent + descent`, with the ink placed so the baseline lands at `y = ascent`. `bitmapTx` is the horizontal offset from the pen origin to the left edge of the ink box - usually 0 for ASCII glyphs, negative for italics that overhang to the left.
- **Advance:** read from `GetCharABCWidthsW` when available, falling back to `GetCharWidth32W`. The result is in pixels, converted to `GdiGlyphRaster.advance`.
- **Kerning:** `GetKerningPairsW` is queried lazily on the first `calculateKerning` call and the result is hashed by `(left, right)`. Pairs absent from the table return 0.

4.4 hasKerningInfo policy

The implementation always returns `true`. Returning `false` would force Ruffle to skip `calculateKerning` for every pair, but pairs without a dedicated entry already return 0 - indistinguishable from "no kerning info" for that pair. Truthful self-reporting would require eagerly creating a `HFONT` and reading its kerning table; deferring the work to the first kerning request is cheaper.

4.5 Reported interface (auto-generated by napi-rs)

```
export interface GdiFontMetrics {
  ascent: number;
  descent: number;
  leading: number;
}

export interface GdiGlyphRaster {
  width: number;
  height: number;
  bitmapTx: number;
  advance: number;
  pixels: Buffer; // RGBA, row-major top-down
}

export declare function createRenderer(
  family: string, bold: boolean, italic: boolean,
): GdiFontRenderer;

export declare class GdiFontRenderer {
  constructor(family: string, bold: boolean, italic: boolean);
  hasKerningInfo(): boolean;
  getFontMetrics(sizePx: number): GdiFontMetrics;
  renderGlyph(codePoint: number, sizePx: number): GdiGlyphRaster | null;
  calculateKerning(left: number, right: number, sizePx: number): number;
  destroy(): void;
}
```

Note that this interface is structurally identical to the one in section 3.3, with napi-rs-specific naming (`Buffer` instead of `Uint8Array`, `GdiFontMetrics` for the value type) - it satisfies the `FontBridge` / `CustomFontRenderer` contracts as-is.

4.6 Usage

The addon must be installed on `globalThis.__ruffleCustomFontRenderer` before the first device-font request reaches Ruffle - in an Electron app this is naturally done from a preload script that has `require` access regardless of the renderer's `nodeIntegration` setting:

```
// Ruffle config:
{ deviceFontRenderer: "custom" }

// preload.js - runs before the renderer's page scripts:
const bridge = require("@ruffle-rs/gdi-font-renderer");
globalThis.__ruffleCustomFontRenderer = bridge;
```

Any other mechanism that ends up with the addon object on `globalThis.__ruffleCustomFontRenderer` before the first glyph is rasterized works identically (see Section 3.4).

5. The swash-font-renderer module

swash-font-renderer is the other reference pluggable renderer in this branch. It is a wasm-bindgen WebAssembly module built around the [swash](#) rasterization library - a natural fit when the host is a regular browser and no OS font database is available to query. Font bytes are supplied through `registerFontData`, parsed once with `ttf-parser`, indexed by family/style, and rasterized on demand with hinted grayscale output. As with the GDI module, this is an example of how a pluggable renderer is structured in a given technology, not a requirement imposed by Ruffle.

5.1 Module shape

Item	Value
Crate path	font-renderers/swash-font-renderer
Crate name	ruffle-swash-font-renderer
Build	cargo build --release --target wasm32-unknown-unknown + wasm-bindgen --target web (optional wasm-opt)
Output	pkg/swash-font-renderer.js + pkg/swash-font-renderer_bg.wasm (ESM)
Public exports	registerFontData(source, bytes), createRenderer(family, bold, italic), the CustomFontRenderer wasm-bindgen class
Platforms	Any wasm32 host (browsers, also usable in Node with the corresponding loader)

5.2 Architecture

Single-threaded by construction: the WASM module runs on the main thread. State is kept in a `thread_local!` registry, which avoids the cost of a Mutex and matches the model wasm-bindgen offers.

```
#[derive(Clone, Eq, Hash, PartialEq)]
struct FaceKey { family_lower: String, bold: bool, italic: bool }

#[derive(Clone)]
struct RegisteredFace {
    data: Rc<Vec<u8>>, // shared font blob
    offset: u32,        // face offset (TTC/OTC support)
    has_kern_table: bool,
    units_per_em: u16,
}

thread_local! {
    static REGISTRY: RefCell<HashMap<FaceKey, RegisteredFace>>
        = RefCell::new(HashMap::new());
}
```

5.3 registerFontData - face indexing

Called once per fontSources entry by Ruffle (see Section 2.9). The implementation:

- Calls `ttf_parser::fonts_in_collection` to figure out whether the blob is a single TTF/OTF or a TTC/OTC.
- Iterates every face, parses it with `ttf_parser::Face::parse(bytes, offset)`.

- Pulls the family name from the name table (Unicode entries only), lowercases it, and combines it with the bold/italic flags into a FaceKey.
- Stores a RegisteredFace sharing the underlying Rc<Vec<u8>> with all faces from the same blob - one heap allocation for the blob, regardless of how many faces it contains.
- Logs and returns silently on parse failures: this matches the embedded path's behaviour, which also degrades gracefully.

5.4 createRenderer and the SizedFont cache

```
#[wasm_bindgen]
pub struct CustomFontRenderer {
    face: RegisteredFace,
    charmap_proxy: CharmapProxy,
    metrics_proxy: MetricsProxy,
    scale_context: ScaleContext,
    by_size: HashMap<u32, SizedFont>,
}

struct SizedFont {
    ascent_px: i32,
    descent_px: i32,
    leading_px: i32,
    glyphs: HashMap<u32, Option<CachedGlyph>>,
}

#[derive(Clone)]
struct CachedGlyph {
    width: u32, height: u32,
    bitmap_tx: f32, advance: f32,
    pixels: Vec<u8>,
}
```

createRenderer looks up the requested FaceKey in the registry. A miss returns None (i.e. JS null), which Ruffle interprets as "the bridge does not handle this family" and falls back to the canvas path. A hit produces a fresh CustomFontRenderer with its own ScaleContext (swash's per-thread rasterization scratch space) and an empty per-size cache.

getFontMetrics(sizePx) populates by_size[size_px] on first call by querying MetricsProxy, scaling by size_px / units_per_em, and rounding.

5.5 renderGlyph - rasterization with bitmap padding

swash returns a tight bitmap whose origin is the glyph's vector origin (baseline, left side of the ink box). Ruffle's consumer expects a line-box bitmap where height = ascent + descent and the baseline sits at row y = ascent. The implementation pads the swash output vertically to satisfy the convention - this was the bug fix that resolved the early "text appears too high in the line box" issue.

```
fn render_glyph_entry(
    scale_context: &mut ScaleContext,
    font_ref: &FontRef<'_>,
    glyph_id: u16,
    ppem: f32,
    ascent_px: i32,
    descent_px: i32,
) -> Option<CachedGlyph> {
```

```

let mut scaler = scale_context
    .builder(*font_ref)
    .size(ppem)
    .hint(true)
    .build();

let image = Render::new(&[Source::Outline])
    .format(Format::Alpha)
    .render(&mut scaler, glyph_id)?;

let placement = image.placement; // zero::Placement, BottomLeft origin
let in_w = placement.width as i32;
let in_h = placement.height as i32;
let out_w = in_w.max(1) as u32;
let out_h = (ascent_px + descent_px).max(1) as u32;

// Vertical padding: baseline sits at row `ascent_px`,
// ink spans `placement.top` rows above the baseline.
let dst_y_offset = ascent_px - placement.top;
let mut pixels = vec![0u8; (out_w * out_h * 4) as usize];
for src_y in 0..in_h {
    let dst_y = src_y + dst_y_offset;
    if dst_y < 0 || dst_y >= out_h as i32 { continue; }
    for src_x in 0..in_w {
        let alpha = image.data[(src_y * in_w + src_x) as usize];
        let i = ((dst_y as u32 * out_w + src_x as u32) * 4) as usize;
        // Premultiplied white
        pixels[i + 0] = alpha;
        pixels[i + 1] = alpha;
        pixels[i + 2] = alpha;
        pixels[i + 3] = alpha;
    }
}

Some(CachedGlyph {
    width: out_w, height: out_h,
    bitmap_tx: placement.left as f32,
    advance: scaler.advance(glyph_id).x,
    pixels,
})
}

```

Quality settings: **full hinting** is requested via `.hint(true)` and **grayscale alpha** (`Format::Alpha`) is requested rather than subpixel - the result is portable across screens of any orientation/subpixel layout, which matches what a generic Flash player needs to look right on any user's setup.

5.6 calculateKerning

Kerning is read directly from the OpenType kern table via `ttf-parser`, iterating every subtable until a pair matches. The result, in font units, is scaled by `size_px / units_per_em` and returned in pixels. `hasKerningInfo()` honestly reports whether the parsed face has a kern table, so Ruffle can skip the per-pair calls when there is no point.

5.7 Lifecycle and memory

- The `Rc<Vec<u8>>` blob is shared between the registry and every renderer derived from it; dropping a renderer alone does not free the blob.

- `destroy()` is exposed but is essentially a no-op in WASM - the per-size cache is freed when the `wasm-bindgen` object is GC'd. It exists to satisfy the contract symmetry with the `napi-rs` side.
- Calling `registerFontData` twice for the same source is idempotent: identical `FaceKeys` overwrite the entry. This lets the host re-register fonts without leaking the previous `Rc`.

5.8 Usage

The `postbuild.mjs` script that accompanies the crate rewrites the `wasm-bindgen` glue so that the module self-initialises via top-level `await`. The bridge methods (`createRenderer`, `registerFontData`) are plain named exports, so the whole module namespace object satisfies the `FontBridge` contract - a single `await import(...)` produces a value that can be assigned to the global slot as-is:

```
<script type="module">
  globalThis.__ruffleCustomFontRenderer =
    await import("swash-font-renderer.js");
</script>
<script src="ruffle.js"></script>
<script>
  window.RufflePlayer.config = {
    deviceFontRenderer: "custom",
    fontSources: ["tahoma.ttf", "tahomabd.ttf"],
  };
</script>
```

The two subsequent classic `<script>` tags load Ruffle and assign the configuration. Script ordering here is convenient but not critical: the browser blocks execution of the classic `ruffle.js` tag until the preceding module script has finished its top-level `await`, so by the time the Ruffle player is instantiated the bridge is already in place. Even if that were not the case, the lookup in `load_device_font` happens per font request (Section 2.4), so a bridge that arrives later would still be picked up before the first glyph is rendered.

Failure of the import is non-fatal for the page as a whole: if the module script throws, the global slot is simply never assigned, and when the first device-font request arrives the Rust side logs a warning and substitutes the canvas renderer for that request (Section 2.4). The user sees the page render, just with browser-rasterized text instead of the swash-rasterized text. Wrapping the `await import(...)` in `try / catch` is advisable in production so that diagnostics can be logged, but it is not required for correctness.

Appendix A. Future work: the JS / WASM boundary

Every pluggable renderer call - `renderGlyph`, `calculateKerning`, `getFontMetrics` - crosses the WASM / JavaScript boundary twice: Ruffle's WASM module calls into the bridge object through `js_sys::Reflect`, the bridge dispatches to the backend, and the result is marshaled back across the same boundary. This is true **independently of whether the backend is a napi-rs native addon (GDI) or another WASM module (swash)**: both reach Ruffle through the same JS-side bridge object, so the marshaling cost is paid in both topologies.

A.1 What each boundary crossing costs

Per call, there is enough overhead that it becomes visible in profiles when the cache is cold:

- Copying primitive arguments (code point, size in pixels, bold / italic flags) through `wasm-bindgen`'s generated glue.
- For `renderGlyph`, allocating a JS object with five fields (width, height, `bitmapTx`, `advance`, `pixels`) and a `Uint8Array` view of the pixel buffer.
- Reading those fields back from Rust via `Reflect::get`, one call per field, with an `Option<JsValue>` round trip each time.
- Copying the `Uint8Array` contents into a `Vec<u8>` so Rust can own the pixels.

The swash backend avoids the last step slightly - its `pixels` view aliases WASM linear memory directly - but the other four items are identical. In particular, the reflective property lookups on the returned object are the same for GDI and swash.

A.2 Why it is not a problem today

The Rust-side cache introduced in Section 1.3 already amortizes the cost: every (`code_point`, `height_px`) bitmap crosses the boundary **exactly once**, when the layout first asks for it. Subsequent frames, retraced strings, and even most re-layouts with the same runs hit the cache and never touch JavaScript. Metrics are similarly cached at the canonical size on renderer creation, and kerning is cached per pair per size once computed. A typical Flash movie in steady state is rendered with essentially no boundary traffic - it is the warm-up phase of a new font size that dominates, and that is bounded by the glyph set the movie actually uses.

A.3 When it might be worth optimizing

If profiling ever shows measurable boundary overhead - for instance a frame-time spike when a long, glyph-diverse text field is first displayed, or a movie that cycles through many different font sizes - the following directions are worth considering. They are listed roughly in order of expected effort vs. benefit:

- **Batch glyph requests.** Extend the bridge with a `renderGlyphs(codes: Uint32Array, sizePx: number)` entry point that returns an array of glyph descriptors in one JS call. A single boundary crossing for every unique code point in a newly visible string would collapse the per-glyph overhead into a per-string overhead.
- **Shared memory for pixel buffers.** Have Rust preallocate a scratch `ArrayBuffer` sized to the largest expected glyph bitmap and hand it to the bridge for in-place writes. This removes the `Vec<u8>`-side copy

and the per-glyph Uint8Array allocation.

- **Bulk kerning queries.** For fonts with dense kerning tables, expose a `getKerningTable(sizePx)` hook that returns the whole table once and lets Rust index it locally, instead of calling across the boundary for every adjacent pair.
- **Flat struct ABI.** Drop the object-return convention in favour of a packed `Int32Array` whose known slots carry the metrics fields - no `Reflect::get` lookups, just array indexing. The existing object-based API would be kept as a compatibility mode for older bridges.

Why deferred: each of these requires extending the public bridge contract in Section 2.6 and the corresponding TypeScript surface in `custom-font-bridge.ts`, which a third-party addon could not ignore without losing functionality. Introducing them speculatively would lock in an API shape before real workloads tell us what the hot path looks like. The current cache-first design keeps the bridge narrow and the hot path cold; these optimizations should be revisited only once there is a concrete performance signal to validate them against.